



Chapter

Architectures

Two



Contents

- ❖ Introduction
- ❖ Architectural Styles
- ❖ System Architectures



Introduction

- ❖ The organization of distributed systems is mostly about the software components that constitute(form) the system. These **software architectures** tell us **how the various software components are to be organized and how they should interact.**
 - ❖ The actual realization of a distributed system requires that we instantiate and place software components on real machines. There are many different choices that can be made in doing so. The final instantiation of a software architecture is also referred to as **a system architecture.**
-



Architectural Styles

- ❖ Distributed systems are often complex pieces of software of which the components are by definition dispersed across multiple machines.
 - ❖ To master their complexity, it is crucial that these systems are properly organized.
 - ❖ Using components and connectors, we can come to various configurations, which, in term have been classified **into architectural styles**.
 - ❖ A component is a modular unit with well-defined required and provided interfaces that is replaceable within its environment.
-



Introduction

The Key Difference

Aspect	Software Architecture Styles	System Architectures
Focus	Logical organization of components	Physical deployment of components
Examples	Layered, Object-based, Data-centered, Event-based	Centralized, Multitiered, Decentralized
Concern	How modules interact and share data	How services are distributed across machines
Abstract vs Concrete	Abstract / Design-level	Concrete / Deployment-level



Con'd

❖ Several styles have by now been identified, of which the most important ones for distributed systems are:

1. Layered architectures
2. Object-based architectures
3. Data-centered architectures
4. Event-based architectures



Con'd

- ❖ The basic idea for the layered style is simple: **components are organized in a layered fashion** where a component at layer L_i is allowed to call components at the underlying layer L_{i-1} ; but not the other way around.
- ❖ A key observation is that control generally flows from layer to layer: **requests go down the hierarchy whereas the results flow upward.**
- ❖ A far looser organization is followed in **object-based architectures**, which are illustrated in Fig. 2-1(b).
- ❖ In essence, each object corresponds to what we have defined as a component, and these components are connected through a (remote) procedure call mechanism.
- ❖ The layered and object based architectures still form the most important styles for large software systems

Con'd

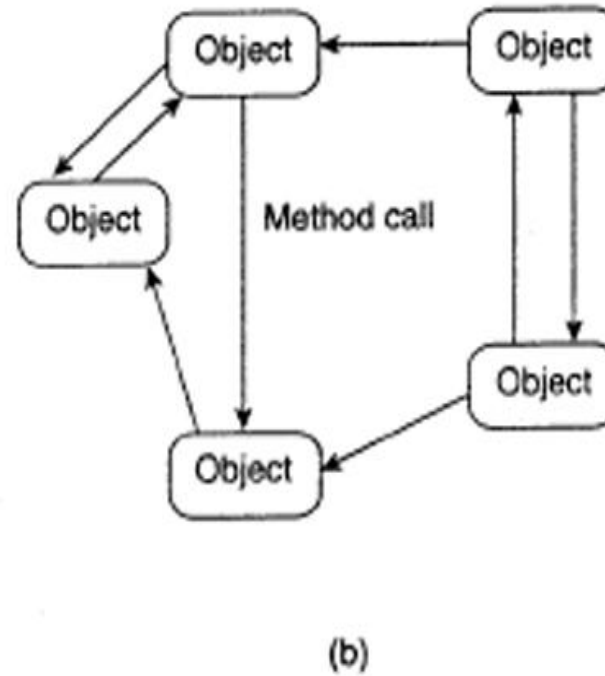
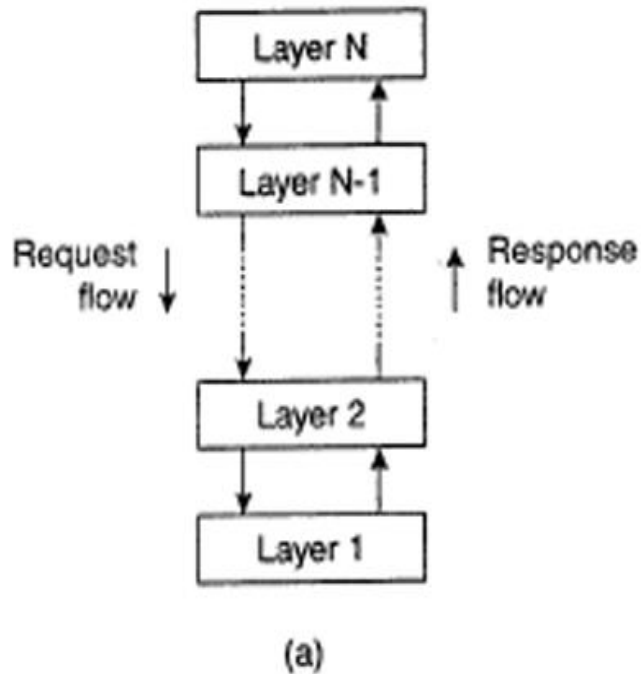


Figure 2-1. The (a) layered and (b) object-based architectural style.

Example for Object-based
Imagine an online shopping system
with objects like

- User object
- Order object
- Payment object
- Inventory object

Flow example

User object calls Order object to
create order

Order object calls Inventory object
to check stock

Order object calls Payment object to
process payment

Each object only focuses on its own
responsibility.

Request flow (downward) for Layered e-commerce
User action → Presentation → Business logic → Data access →
Database



Con'd

- ❖ **Data-centered architectures** evolve around the idea that processes communicate through a common (passive or active) repository.
- ❖ It can be argued that for distributed systems these architectures are as important as the layered and object based architectures.
- ❖ **In event-based architectures**, processes essentially communicate through the propagation of events, which optionally also carry data, as shown in Fig. 2-2(a).
- ❖ For distributed systems, event propagation has generally been associated with what are known as publish/subscribe systems.
- ❖ Event-based architectures can be combined with data-centered architectures, yielding what is also known as **shared data spaces**.
- ❖ The **essence of shared data** spaces is that **processes** are now also decoupled in time: they need not both be active when communication takes place.

Con'd

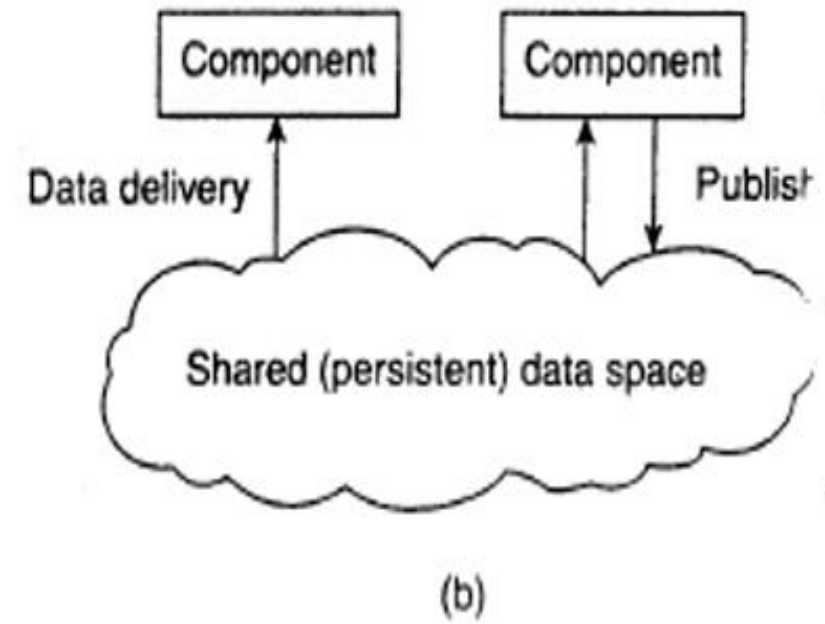
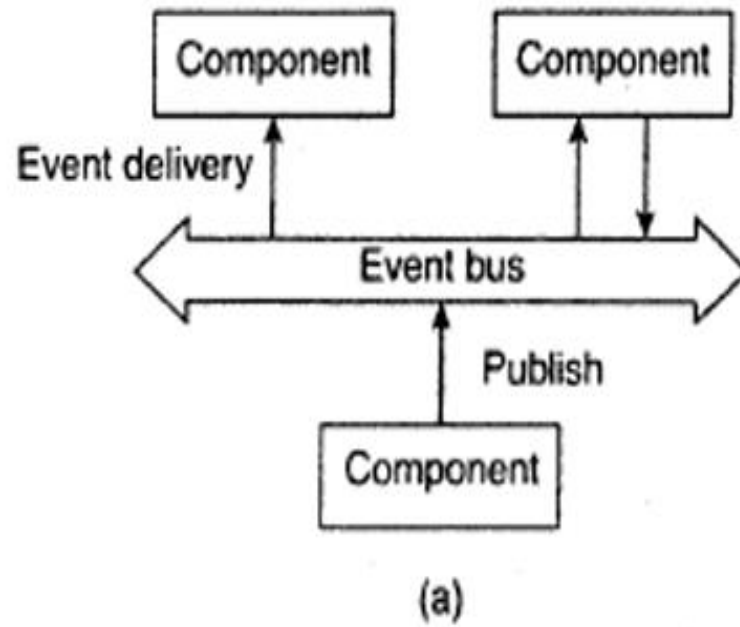


Figure 2-2. The (a) event-based and (b) shared data-space architectural style.



System Architectures

Centralized Architectures

- ❖ In the basic client-server model, processes in a distributed system are divided into two (possibly overlapping) groups.
 - ❖ A **server** is a process implementing a specific service, for example, a file system service or a database service.
 - ❖ A **client** is a process that requests a service from a server by sending it a request and subsequently waiting for the server's reply. This client-server interaction, also known as request-reply behavior is shown in Fig. 2-3.
-

Con'd

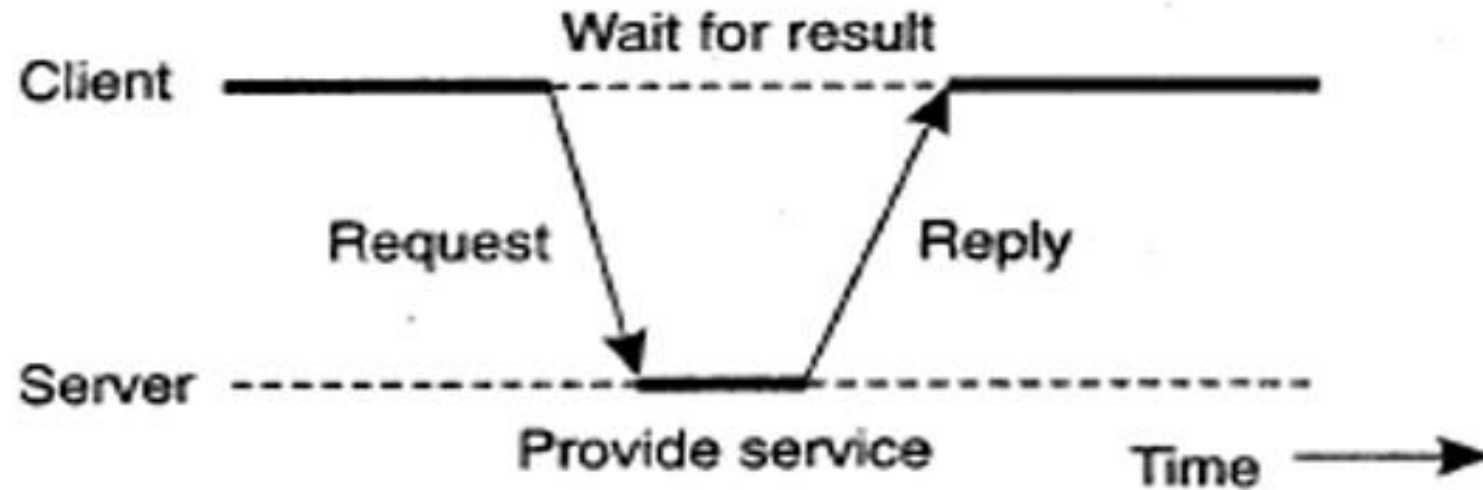


Figure 2-3. General interaction between a client and a server.



Con'd

- ❖ Communication between a client and a server can be implemented by means of a simple connectionless protocol when the underlying network is fairly reliable as in many **local-area networks**.
 - ❖ In these cases, when a client requests a service, it simply packages a message for the server, identifying the service it wants, along with the necessary input data.
 - ❖ The message is then sent to the server. The latter, in turn, will always wait for an incoming request, subsequently process it, and package the results in a reply message that is then sent to the client.
-



Con'd

- ❖ As an alternative, many client-server systems use a **reliable connection oriented protocol**.
 - ❖ Although this solution is not entirely appropriate in a **local-area network** due to relatively low performance, it works perfectly fine in wide-area systems in which communication is inherently unreliable.
 - ❖ For example, virtually all Internet application protocols are based on reliable TCP-IP connections.
 - ❖ In this case, whenever a client requests a service, it first sets up a connection to the server before sending the request.
 - ❖ The server generally uses that same connection to send the reply message, after which the connection is torn down.
-



Application Layering

- ❖ There is often no clear distinction b/n client and server.
- ❖ For example, a server for a distributed database may continuously act as a client because it is forwarding requests to different file servers responsible for implementing the database tables.
- ❖ Many people have advocated a distinction between the following three levels, essentially following the layered architectural style we discussed previously:
 - ❖ 1. **The user-interface level:** contains interface with the user
 - ❖ 2. **The processing level:** contains the applications
 - ❖ 3. **The data level:** manages the actual data

Con'd

- ❖ The simplified example of their organization is an Internet search engine into three different layers.

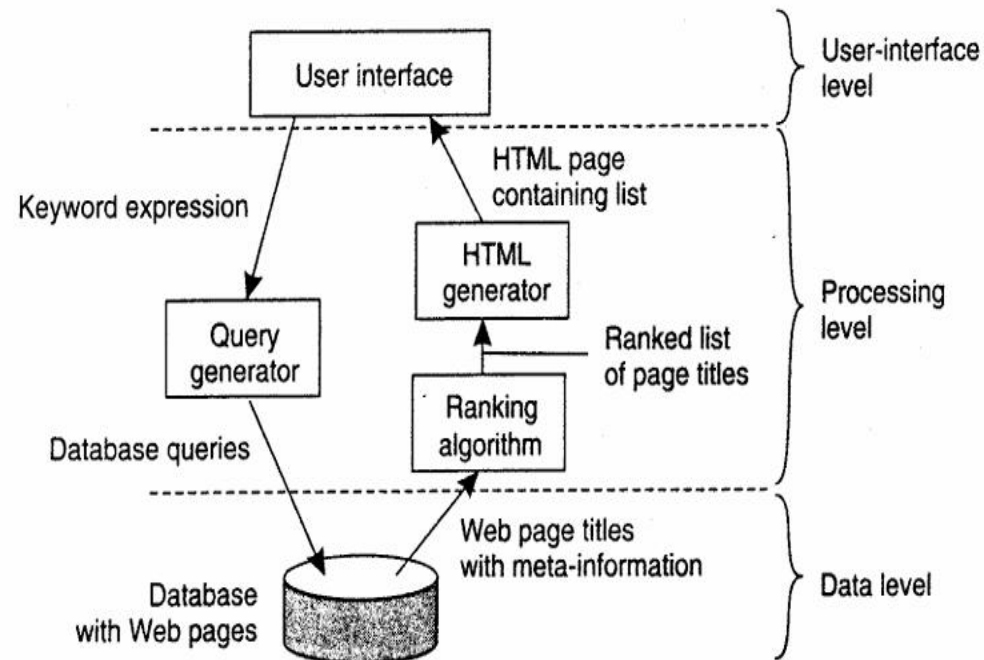


Figure: The simplified organization of an Internet search engine into three different layers.



Multitiered Architectures

- ❖ The simplest organization is to have only two types of machines:
 - ❖ 1. A client machine containing only the programs implementing (part of) the **user-interface level**.
 - ❖ 2. A server machine containing the rest, that is the programs implementing the processing and data level.
 - ❖ We refer to this type of distribution as vertical distribution. The characteristic feature of vertical distribution is that it is achieved by placing logically different components on different machines.
-

Multitiered Architectures

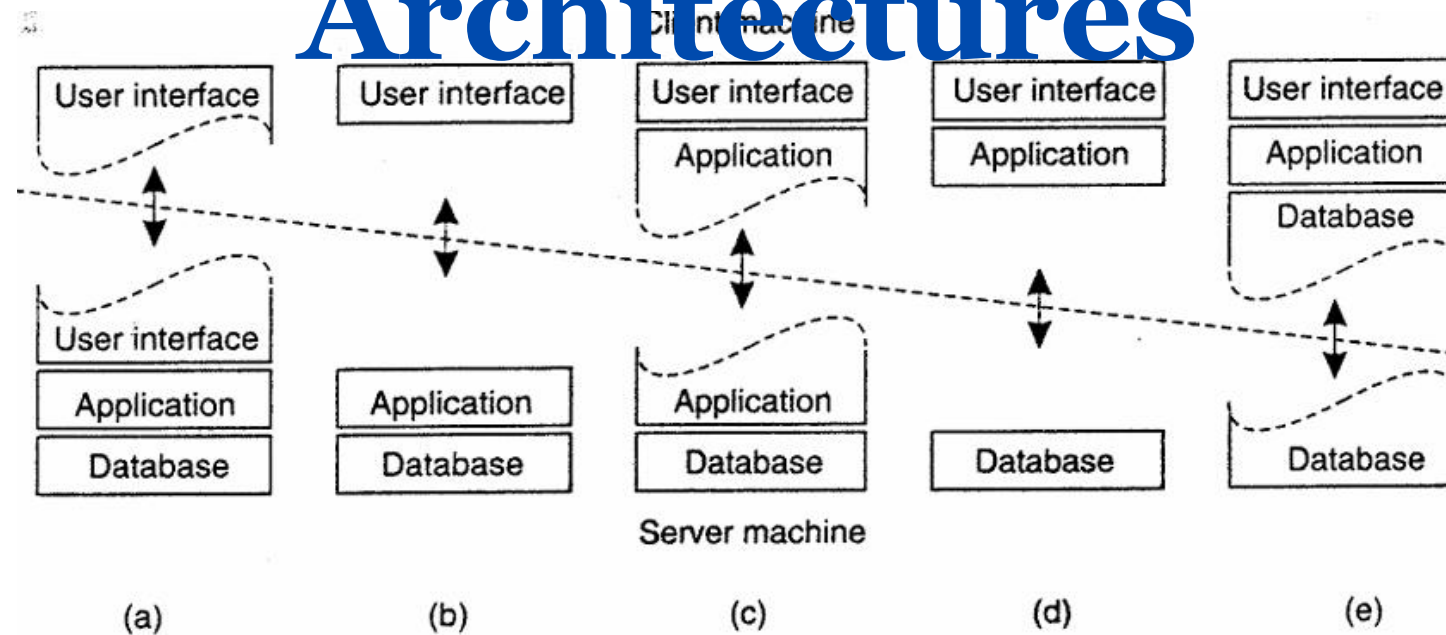


Figure: Alternative client-server organizations (a)-(e).

Con'd

- ❖ Server may sometimes need to act as a client,
- ❖ A typical example of where a **three-tiered architecture** is used in transaction processing.
- ❖ A separate process, called the transaction processing monitor, coordinates all transactions across possibly different data servers.

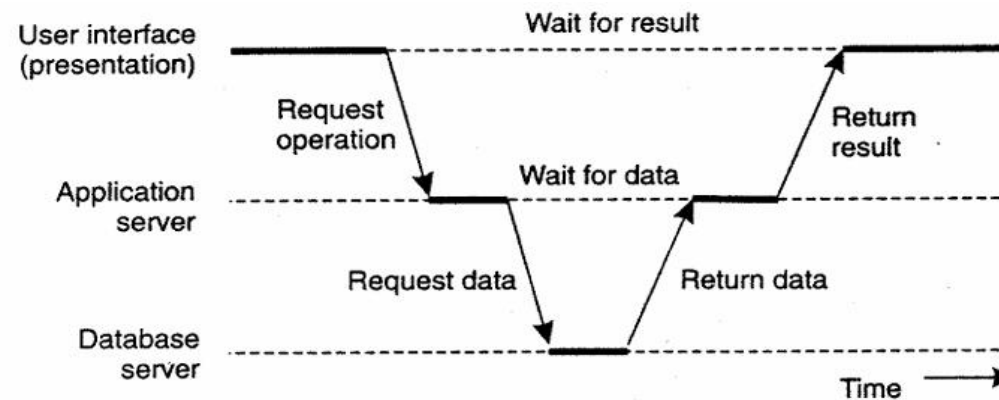


Figure: An example of a server acting as client.



Decentralized Architectures

- ❖ we refer to as horizontal distribution which is known as **peer-to-peer systems**.
- ❖ Interaction between processes is symmetric: each process will act as a client and a server at the same time (which is also referred to as acting as a servant).
- ❖ A process cannot communicate directly with an arbitrary other process, but is required to send messages through the available communication channels.
- ❖ Two types of overlay networks exist: those that are **structured** and **those that are Unstructured Peer-to- Peer Architectures**.



Structured peer-to-peer Architectures

- ❖ Overlay network is constructed using a deterministic procedure.
- ❖ Organize the processes through a distributed hash table (DHT).
- ❖ The DHT decides
 - ❖ which node stores which data
 - ❖ how to find the node responsible for a key
- ❖ In a DHT -based system, data items are assigned a random key from a large identifier space, such as a 128-bit or 160-bit identifier.
- ❖ If keys are 128 bit :There are 2^{128} possible numbers
- ❖ That is an extremely huge range. Why so big
 - ❖ • to avoids two data items getting the same key
 - ❖ • to allows millions or billions of nodes and files



Unstructured peer-to-peer Architectures

- ❖ Largely rely on randomized algorithms for constructing an overlay network.
- ❖ Data items are assumed to be **randomly placed on nodes**.
- ❖ As a consequence, when a node needs to locate a specific data item, the only thing it can effectively do is flood the network with a search query.
- ❖ One of the goals is to construct an overlay network that resembles a random graph.
- ❖ The basic model is that each node maintains a list of neighbours, where, ideally, each of these neighbours represents a randomly chosen live node from the current set of nodes.



Topology management of overlay networks

- ❖ One key observation is that by carefully exchanging and selecting entries from partial views, it is possible to construct and maintain specific topologies of overlay networks.
- ❖ Each node does not know all nodes in the network.
- ❖ It only keeps a small list of known nodes.
- ❖ This list is called a partial view or neighbor list.
- ❖ Nodes periodically share their neighbor lists with each other.
- ❖ So Node A sends some of its neighbors to Node B
- ❖ Node B merges and selects some of them
- ❖ By repeating this process across the network
- ❖ The overall topology gradually forms



Super Peers

- ❖ Super peers are a concept used in hybrid peer to peer architectures, where some nodes take on extra responsibilities to improve performance and scalability.
 - ❖ Instead of every node being equal, the network has
 - ❖ • regular peers
 - ❖ • more powerful nodes called super peers
 - ❖ Organized in a peer-to-peer network, leading to a hierarchical organization.
-



Hybrid Architectures

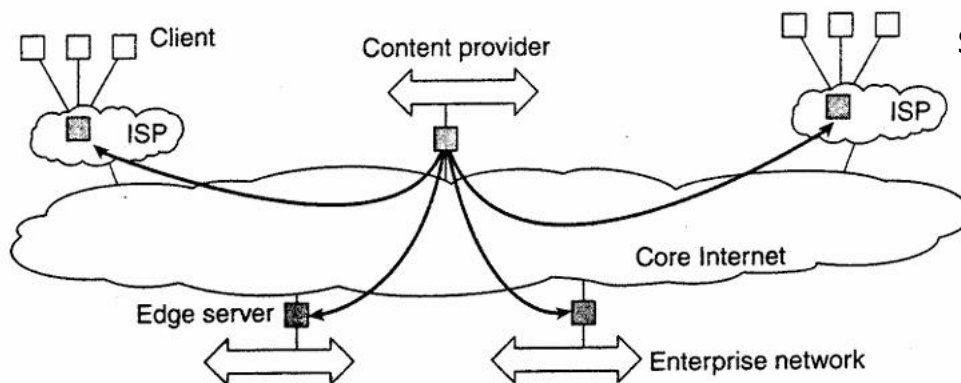
- ❖ Distributed systems in which client-server solutions are combined with decentralized architectures.

Edge-Server Systems

- ❖ Servers are placed at the edge of the network formed by enterprise networks and the actual Internet, for example, as provided by an Internet Service Provider (ISP).

Components in the Diagram

- **Client:** End users requesting data or services.
- **ISP (Internet Service Provider):** Provides connectivity between clients and servers.
- **Content Provider:** Central servers hosting the main data or services.
- **Edge Server:** Servers deployed closer to clients, at the “edge” of the network.
- **Enterprise Network:** Internal networks of organizations that may also connect to edge servers.
- **Core Internet:** The backbone network connecting ISPs, content providers, and edge servers.





Collaborative Distributed Systems

- ❖ Traditional client-server scheme is deployed.
- ❖ Once a node has joined the system, it can use a fully decentralized scheme for collaboration.
- ❖ BitTorrent file-sharing system: An important design goal was to ensure collaboration



**Thank
You**
